MAP

학습 지도와 분석 범위

무엇을, 어떤 신뢰 수준으로, 어떤 순서로 읽을 것인가

분석 대상
v2.1.88 reconstructed source

고정 리비전
2ca5ddabfed5

독자
전체 스터디 참여자

사용 방식
오리엔테이션 · 범위 합의

> 교육·연구용 파생 분석입니다. Anthropic의 공식 소스·공식 아키텍처 문서가 아닙니다.

이 자료의 정체와 사용 한계

이 묶음은 “공개된 제3자 복원 저장소를 학습 대상으로 삼은 파생 해설서”다. 공식 구현 사양이나 재배포 가능한 소스 묶음이 아니다.

증거 등급

코드 위치와 분포 수치는 고정 커밋의 정적 분석 결과다. 제품 의도·운영 동작·현재 버전에 대한 설명은 별도 검증 없이는 추정으로 취급한다. 저장소 자체가 “decompiled source”라고 밝히며 완전 재빌드 불가와 누락 모듈을 명시한다.

확인됨

버전·리비전

저장소 표기는 v2.1.88, 분석 고정 커밋은 `2ca5ddabfed5`이다.

확인됨

불완전성

README/QUICKSTART는 feature gate 뒤의 108개 모듈과 내부 빌드 구성이 빠졌다고 설명한다.

주의

라이선스

루트에 LICENSE 파일이 없다. README의 교육·비상업 안내를 법적 라이선스로 오인하지 않는다.

사용 원칙

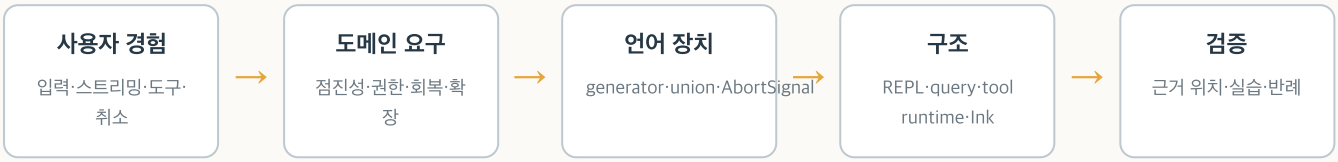
재현보다 이해

코드를 복제하지 않고 메커니즘, 구조, 트레이드오프, 검증 질문을 학습한다.

근거 위치 README_KR.md · QUICKSTART.md · 대상 저장소

요청을 일곱 권으로 분해한 기준

기능 흐름만 좇지 않고 “도메인 요구 → 언어 메커니즘 → 프로그래밍 표현 → 코드 구조 → 운영 속성”을 한 묶음으로 읽는다.



권	핵심 질문	주요 산출
00	어디까지, 어떤 순서로 볼까?	범위·신뢰도·학습 경로
01	시스템은 어떤 책임 덩어리로 이루어졌나?	코드 지도·의존성·핫스폿
02	왜 이런 TypeScript/JavaScript 표현이 필요한가?	언어 메커니즘 해설
03	모델·스트림·도구가 어떻게 한 턴을 만든가?	에이전트 루프 상태기계
04	터미널 UI가 왜 React/Ink인가?	렌더러·입력·스크롤백
05	도구 능력을 어떻게 제한·확장하는가?	권한·훅·MCP·플러그인
06	어떻게 직접 검증하며 공부할까?	실습·퀴즈·평가 루브릭

아홉 개 분석 범주

영상이 강조한 두 축—async generator 에이전트 루프와 React/Ink UI—에 더해 실제 제품을 성립시키는 주변 메커니즘을 포함했다.

1
진입·표면
CLI, interactive REPL, print/headless, SDK, bridge/remote.

2
대화·상태
메시지 union, AppState, 세션 저장, 히스토리, 첨부.

3
모델 호출
요청 구성, 스트리밍 이벤트, fallback, 토큰·비용.

4
에이전트 루프
컨텍스트 압축, 도구 탐지·실행, 후속 턴, 종료 이유.

5
도구 런타임
스키마, 동시성 안전성, 결과 정규화, 진행 이벤트.

6
권한·안전
mode, 규칙, hook, shell AST, sandbox, human gate.

7
터미널 UI
reconciler, Ink, Yoga, ANSI, Static, input hook.

8
확장
MCP, plugin, skill, command, agent/task, tool search.

9
빌드·관측
Bun feature gate, dynamic import, analytics, profiler, 복원 한계.

사실·해석·추정을 섞지 않는 법

복원 소스는 풍부하지만 공식성·완전성이 낮다. 각 주장에 근거 등급을 붙이면 흥미로운 서사를 사실처럼 굳히는 오류를 줄일 수 있다.

등급	정의	예시	보고서 표기
A	고정 커밋의 코드·구조로 직접 확인	`query()`가 `yield*`로 내부 루프를 위임	“코드에서 확인”
B	저장소 문서가 주장하나 독립 검증이 부족	누락 모듈 수, 추출 방식	“저장소 설명에 따르면”
C	공식 사양·공식 문서와 코드가 합치	MCP tool schema, AbortSignal	“사양과 구현이 대응”
D	구조에서 합리적으로 도출한 해석	UI/core 교체 가능성, 특정 trade-off	“해석/가설”
X	현 리비전으로 검증 불가	서버 내부 정책, 제거된 feature의 실제 동작	“미확인”

읽기 규칙

“파일이 존재한다”와 “운영에서 활성화된다”는 다른 주장이다. feature gate, 설정, 서버 capability, 사용자 권한까지 닫혀야 실제 경로가 된다.

두 영상과 보고서의 연결

영상은 좋은 진입점이지만 설명 모델과 정확한 구현 사실을 구분해 읽는다.

영상의 렌즈	핵심 주장	확장·보정할 지점	연결 권
async generator 영상	루프가 await와 yield를 함께 써야 스트림·도구·중단을 표현할 수 있다.	실제 코드는 압축·fallback·tombstone·hook·budget·terminal reason까지 가진다.	02, 03
React/Ink 영상	UI는 React, core는 순수 TS이며 callback 경계로 연결된다.	배칭을 “고정 16ms”로 단정하지 말고 renderer/scheduler 구현과 측정으로 검증한다.	01, 04

좋은 재구성 질문

“제너레이터가 쓰였다”에서 멈추지 말고, 어떤 요구가 iterator의 pull semantics·중단·오류 전파·상태 보존을 필요로 했는지 역으로 추적한다.

상위 흐름에서 세부 표현까지 내려가기

모든 장은 동일한 다섯 층을 왕복한다. 이 방식이 단순 호출 그래프를 교육 자료로 바꾸는 핵심이다.

층	질문	대표 개념	코드에서 찾을 표현
도메인	사용자가 무엇을 기대하는가?	응답이 즉시 보임, 취소 가능, 위험 작업 확인	입력·progress·permission UI
시간	무엇이 언제 도착하고 멈추는가?	SSE, backpressure, turn, retry	`async function*`, `for await`, signal
타입	유효한 상태를 어떻게 표현하는가?	event algebra, terminal reasons	discriminated union, `never`
구조	변경 이유를 어디에 가두는가?	renderer adapter, tool contract	module, interface, composition
운영	실패·권한·관측을 어떻게 다루는가?	least privilege, complete mediation	hook, permission, telemetry, checkpoint

코드 읽기 순서: 바깥에서 안쪽으로

대형 파일부터 무작정 읽지 않는다. 먼저 안정된 계약과 경계를 잡고, 한 사용자 사건을 따라 내부로 들어간다.

1. **정체성·한계** — README_KR.md, QUICKSTART.md.
2. **메시지·권한 타입** — 복원본에서 빠진 `types/message`의 참조 지점 src/query.ts과 src/types/permissions.ts.
3. **도구 계약** — src/Tool.ts와 `buildTool`.
4. **루프 골격** — src/query.ts → 650 → 1357 → 1715.
5. **실행 파이프** — src/services/tools/toolOrchestration.ts와 toolExecution.
6. **UI 경계** — src/screens/REPL.tsx와 PromptInput.
7. **renderer 내부** — `src/ink/reconciler.ts`, `renderer.ts`, `render-to-screen.ts`.
8. **확장·보안** — permissions, hooks, MCP, plugins, skills를 관심사별로 읽는다.

> **중단 기준**
>
> 30분 안에 “이 파일이 어떤 경계를 소유하는가”를 한 문장으로 말할 수 없으면 호출자를 더 읽지 말고 타입·public contract·데이터 모양으로 돌아간다.

상세 보고서를 위해 추가한 주제

요청에 직접 언급되지 않았지만 기능의 이유와 구조를 연결하려면 빠뜨리기 어려운 분석 영역이다.

시간

취소·timeout·retry

비동기 흐름의 품질은 성공 경로보다 중단 경로에서 드러난다.

메모리

압축·결과 예산

긴 세션에서 컨텍스트가 유한 자원이라는 사실이 루프 구조를 바꾼다.

능력

권한·sandbox

도구가 생기는 순간 모델 출력은 외부 효과 권한으로 바뀐다.

확장

MCP·plugin·skill

모든 확장은 같은 수준이 아니며 discovery, schema, execution authority가 다르다.

관측

telemetry·checkpoint

복잡한 turn을 디버깅하려면 단계·원인·식별자를 구조화해야 한다.

구조 부채

의존성 순환·허브

복원된 1,884파일에서 거대 SCC가 보이며, 변경 위험을 따로 읽어야 한다.

권장 스터디 운영

주 2회, 회당 90분을 기준으로 “개념 25분 → 코드 35분 → 실험 20분 → 회고 10분”의 리듬을 사용한다.

1주 · 증거와 지도
00-01권, repo 신뢰도·entry surface·핫스폿

2주 · 비동기 언어
02권, iterator/generator/AbortSignal 미니 실험

3주 · 에이전트 턴
03권 전반, query state·stream·tool-use

4주 · 회복과 컨텍스트
03권 후반, compaction·fallback·terminal reason

5주 · 터미널 UI
04권, React reconciler·Ink·prompt submit

6주 · 도구·권한
05권 전반, schema·permission·hook·shell safety

7주 · 확장 시스템
05권 후반, MCP·plugin·skill·tool search

8주 · 종합 실습
06권, trace·threat model·architecture review

> **운영 팁**
> 매 회차 한 명은 “설명자”, 한 명은 “반례 찾는 사람”, 한 명은 “근거 링크 관리자”를 맡는다. 역할은 다음 회차에 순환한다.

학습 결과물의 완료 조건

"읽었다"가 아니라 재구성·검증·설계 전이 능력으로 완료를 판단한다.

- ☐ 한 사용자 입력이 UI와 core에서 갈라지는 경계를 그릴 수 있다.
- ☐ async generator가 Promise>보다 나은 요구를 세 가지 설명할 수 있다.
- ☐ tool schema, permission, execution, result normalization을 구분할 수 있다.
- ☐ permission mode와 개별 tool의 read-only/concurrency 속성을 혼동하지 않는다.
- ☐ React와 Ink, reconciler와 renderer의 역할을 분리해 말할 수 있다.
- ☐ 컨텍스트 압축이 단순 문자열 요약이 아니라 루프 상태 전이임을 설명한다.
- ☐ feature gate가 "코드 존재 = 기능 활성" 등식을 깨뜨리는 이유를 말할 수 있다.
- ☐ 코드 근거와 공식 사양, 추정을 서로 다른 라벨로 기록한다.
- ☐ 실패·취소·재시도·권한 거부의 terminal evidence를 설계할 수 있다.
- ☐ 다른 agentic CLI에 이 분석 틀을 재사용할 수 있다.

실습 · 1분 설명

"Claude Code는 React 앱이다"라는 말을 정확하고 과장 없이 120자 이내로 다시 쓴다.

완료 증거: 경계·renderer·core 세 단어가 포함된 문장

시작 전에 맞출 핵심 어휘

같은 단어를 다른 수준에서 쓰면 토론이 흐려진다. 이 묶음에서는 아래 의미로 고정한다.

용어	이 자료의 의미	혼동 주의
turn	한 번의 사용자 요청에서 모델·도구 후속 호출이 수렴하는 범위	API request 한 번과 다름
stream event	시간에 따라 점진적으로 도착하는 관찰 단위	최종 assistant message와 다름
tool	스키마와 실행·권한·표시 계약을 가진 capability	shell command 하나와 다름
hook	lifecycle 지점에서 실행되는 확장·정책 처리	권한 자체와 다름
permission	특정 효과가 실행되어도 되는지의 결정	tool discoverability와 다름
renderer	React tree의 차이를 특정 출력 매체에 적용하는 adapter	reconciler와 다름
compaction	유한 컨텍스트 예산 안에서 대화 표현을 재구성하는 과정	파일 압축이 아님
feature gate	빌드·런타임 조건에 따라 경로를 포함/배제하는 선택	파일 존재 여부가 아님

이제 어디로 갈까

처음 읽는다면 01→02→03→04→05→06 순서를 권한다. 특정 관심사가 있다면 아래 분기를 따른다.

아키텍처

01권부터

entrypoint, 책임 지도, 의존성 hotspot을 먼저 잡는다.

언어

02권부터

generator, union, generic, cancellation을 집중해서 본다.

AGENT RUNTIME

03권부터

실제 query loop의 상태·회복·도구 후속 턴을 읽는다.

FRONTEND/TUI

04권부터

React와 terminal renderer의 경계를 추적한다.

SECURITY/PLATFORM

05권부터

capability, permission, hook, MCP, plugin을 본다.

HANDS-ON

06권부터

실습 카드를 뽑고 필요한 권으로 역참조한다.

권장 원칙

첫 회독에는 모든 파일을 열지 않는다. 보고서가 지시한 anchor line을 읽고, 질문이 생긴 경우에만 인접 파일로 확장한다.